도메인 설계

1. 도메인 설계

프로젝트의 핵심 비즈니스 로직을 정의하는 단계입니다.

1.1. 프로젝트 개요

- **프로젝트명: 취미/스터디 그룹 매칭 플랫폼 프로젝트**
- **도메인 영역**: 소셜 네트워킹, 그룹 매칭
- **핵심 비즈니스**: 사용자들이 관심사(취미, 스터디)에 맞는 그룹을 생성, 검색, 가입하고 활동할 수 있는 플랫폼 제공

1.2. 핵심 도메인 객체

`USER` (회원)

- **설명**: 서비스를 이용하는 사용자. 프로필, 생성/가입한 그룹, '찜'한 그룹 등의 정보를 가집니다.
- **주요 속성**: 사용자 ID, 이메일, 비밀번호, 닉네임, 지역, 프로필 이미지 URL, 자기소개, 리프레시 토큰, 생성/삭제 일시
- **주요 행위**: 회원가입, 로그인/로그아웃, 토큰 재발급, 내 정보 조회/수정, 회원 탈퇴, 그룹 생성/가입/탈퇴, 그룹 찜하기, 게시글/댓글 작성, 이벤트(일정) 참석

`GROUP` (그룹)

- **설명**: 사용자들이 생성하는 취미 또는 스터디 모임입니다.
- **주요 속성**: 그룹 ID, 그룹명, 소개, 지역, 최대 인원, 현재 인원, 그룹 이미지 URL, 커버 이미지 URL, 태그, 생성자, 카테고리, 생성/수정/삭제 일시
- **주요 행위**: 그룹 생성, 정보 수정, 멤버 관리 (강퇴), 그룹 삭제(논리적), 모임장 위임, 커버 이미지 관리

`CATEGORY` (카테고리)

- **설명**: 그룹의 관심사를 분류합니다 (예: 운동, 개발, 여행).
- **주요 속성**: 카테고리 ID, 카테고리명
- **주요 행위**: 카테고리별 그룹 조회

MEMBERSHIP (멤버십)

- **설명**: 사용자와 그룹 간의 가입 관계(N:M)를 나타내는 중간 객체입니다.
- **주요 속성**: 멤버십 ID, 사용자 ID, 그룹 ID, 가입일, 멤버 등급 (OWNER/MEMBER)
- **주요 행위**: 가입, 탈퇴, 역할 변경

POST (게시글)

- **설명**: 그룹 내에서 작성되는 모든 글 (공지, 일반글 등).
- **주요 속성**: 게시글 ID, 제목, 내용, 이미지 URL, 작성자, 소속 그룹, 생성/수정/삭제 일시
- **주요 행위**: 게시글 작성, 수정, 삭제(논리적), 조회, 이미지 관리

COMMENT (댓글)

- **설명**: 게시글에 대한 사용자 반응. 대댓글 기능을 지원합니다.
- **주요 속성**: 댓글 ID, 내용, 작성자, 원본 게시글, 부모 댓글 ID (대댓글용), 생성/수정/삭제 일시
- **주요 행위**: 댓글/대댓글 작성, 수정, 삭제(논리적), 조회

EVENT (이벤트/일정)

- **설명**: 그룹 내에서 생성되는 정모, 스터디 일정 등.
- **주요 속성**: 이벤트 ID, 제목, 설명, 장소, 이미지 URL, 최대 참석 인원, 시작/종료 시간, 주최자, 소속 그룹, 생성/수정 일시
- **주요 행위**: 이벤트 생성, 조회, 수정, 삭제

EVENT_ATTENDEE (이벤트 참석자)

- **설명**: 이벤트와 사용자 간의 참석 관계(N:M)를 나타내는 중간 객체입니다.
- **주요 속성**: 참석 ID, 이벤트 ID, 사용자 ID, 신청일, 역할(HOST/ATTENDEE), 상태 (CONFIRMED/WAITING)
- **주요 행위**: 참석 신청, 신청 취소, 상태 변경

`GROUP_LIKE` (그룹 찜)

- **설명**: 사용자가 관심 있는 그룹을 '찜'하는 관계(N:M)를 나타냅니다.
- **주요 속성**: 찜 ID, 사용자 ID, 그룹 ID
- **주요 행위**: 찜하기/찜 취소 (Toggle)

1.3. 도메인 관계도

```
erDiagram
    USER ||--o{ GROUP : "생성 (Owner)"
    USER ||--|{ MEMBERSHIP : "가입"
    GROUP ||--|{ MEMBERSHIP : "멤버 보유"
    USER ||--o{ POST : "작성"
    GROUP ||--|{ POST : "게시글 보유"
    POST ||--|{ COMMENT : "댓글 보유"
    USER ||--o{ COMMENT : "작성"
    CATEGORY ||--|{ GROUP : "분류"
    USER ||--o{ EVENT : "주최 (Host)"
    GROUP ||--|{ EVENT : "이벤트 보유"
    USER ||--|{ EVENT_ATTENDEE : "참석"
    EVENT ||--|{ EVENT_ATTENDEE : "참석자 보유"
    USER ||--|{ GROUP_LIKE : "찜"
    GROUP ||--|{ GROUP_LIKE : "찜 보유"
    COMMENT ||--o{ COMMENT : "대댓글"

    USER {
        Long user_id PK
        String email UK
        String password
        String nickname UK
        String region
        String profile_image_url
        String introduction
        String refresh_token
        LocalDateTime created_at
        LocalDateTime deleted_at
    }
    GROUP {
```

```
    Long group_id PK
    String name
    String description
    String region
    int max_members
    String image_url
    String cover_image_url
    Long owner_id FK
    Long category_id FK
}
CATEGORY {
    Long category_id PK
    String name UK
}
MEMBERSHIP {
    Long membership_id PK
    Long user_id FK
    Long group_id FK
    String role
    LocalDateTime joined_at
}
POST {
    Long post_id PK
    String title
    String content
    String image_url
    Long user_id FK
    Long group_id FK
}
COMMENT {
    Long comment_id PK
    String content
    Long user_id FK
    Long post_id FK
    Long parent_id FK
}
EVENT {
    Long id PK
```

```
        String title
        String location
        int max_attendees
        LocalDateTime start_at
        Long host_id FK
        Long group_id FK
    }
    EVENT_ATTENDEE {
        Long id PK
        Long event_id FK
        Long user_id FK
        String role
        String status
    }
    GROUP_LIKE {
        Long group_like_id PK
        Long user_id FK
        Long group_id FK
    }
```

1.4. 비즈니스 규칙 정의

User (회원) 관련 규칙

가입 및 인증

규칙 1: 닉네임은 중복될 수 없다.

- `users` 테이블의 `nickname` 컬럼에 `UNIQUE` 제약조건이 설정되어 데이터베이스 수준에서 중복을 방지합니다.

규칙 2: 이메일 주소는 중복될 수 없다.

- 회원가입 시 서비스 로직에서 먼저 동일한 이메일의 존재 여부를 확인하며, `users` 테이블의 `email` 컬럼에도 `UNIQUE` 제약조건이 있습니다.

규칙 3: 비밀번호는 최소 8자 이상이어야 한다.

- 회원가입 요청 DTO(`SignUpRequest`)에서 `@Size(min = 8)` 어노테이션을 통해 유효성을 검증합니다.

활동

규칙 4: 회원 탈퇴 시, 소유하고 있는 그룹이 있으면 탈퇴할 수 없다.

- 사용자가 탈퇴(`deleteUser`)를 시도할 때, 해당 사용자가 소유한 그룹이 있는지 확인하고, 활성화된 그룹이 하나라도 있으면 탈퇴가 제한됩니다.

Group (그룹) 관련 규칙

생성 및 운영

규칙 1: 그룹명은 중복될 수 없다.

- 그룹 생성 시 데이터베이스에 비관적 락(`PESSIMISTIC_WRITE`)을 걸어 동일한 이름의 그룹이 동시에 생성되는 것을 방지합니다.

규칙 2: 그룹 생성 시, 카테고리를 반드시 선택해야 한다.

- 그룹 생성 요청 DTO(`CreateGroupRequest`)의 `categoryId` 필드에 `@NotNull` 검증이 적용되어 있어 카테고리 선택을 강제합니다.

규칙 3: 그룹장은 자신의 그룹을 탈퇴할 수 없다.

- 그룹 탈퇴(`leaveGroup`) 로직에서, 요청한 사용자의 역할(`role`)이 `OWNER`일 경우 예외를 발생시켜 탈퇴를 막습니다.

규칙 4: 그룹장은 멤버를 강퇴시킬 수 있는 권한을 가진다.

- 멤버 강퇴(`removeMember`) 로직에서, 요청한 사용자가 해당 그룹의 소유주(`owner`)인지 확인하여 권한을 검증합니다.

가입 및 멤버십

규칙 5: Group의 현재 인원이 최대 인원과 같거나 많으면 가입이 불가능하다.

- 그룹 가입(`joinGroup`) 시, 그룹의 현재 멤버 수와 `maxMembers`를 비교하여 정원이 가득 찼으면 `GroupFullException`을 발생시킵니다.

규칙 6: 이미 가입된 그룹에는 중복으로 가입 신청을 할 수 없다.

- 그룹 가입(`joinGroup`) 시, `memberships` 테이블에 해당 사용자와 그룹의 관계가 이미 존재하는지 확인하여 중복 가입을 방지합니다.

Post (게시글) 및 Comment (댓글) 관련 규칙

수정 및 삭제

규칙 1: 게시글과 댓글은 작성자 본인만 수정하거나 삭제할 수 있다.

- 게시글 수정/삭제 시 `validatePostOwner` 메서드가, 댓글 수정/삭제 시 `validateCommentOwner` 메서드가 호출되어 요청한 사용자의 ID와 콘텐츠 작성자의 ID를 비교하여 본인 여부를 확인합니다.

1.5. 상태(State) 정의

Group (그룹)

그룹의 상태(모집 중, 진행 중 등)를 나타내는 별도의 상태 필드는 존재하지 않습니다. 대신 논리적 삭제(Soft Delete)를 통해 그룹의 활성 상태를 관리합니다.

- **ACTIVE (활성)**
  - **상태 설명**: `deleted_at` 필드가 `NULL` 인 상태로, 그룹이 활성화되어 모든 기능(검색, 가입, 게시글 작성 등)을 정상적으로 이용할 수 있습니다.
- **DELETED (삭제)**
  - **진입 조건**: 그룹장이 `DELETE /api/groups/{groupId}` API를 호출했을 때 진입합니다.
  - **상태 설명**: `deleted_at` 필드에 삭제 시간이 기록됩니다. `@Where(clause = "deleted_at IS NULL")` 어노테이션에 의해 삭제된 그룹은 기본적으로 조회되지 않습니다.

Membership (멤버십)

멤버십은 별도의 상태 필드 없이, 데이터베이스 레코드의 존재 여부로 상태가 정의됩니다.

- **CREATED (가입 완료)**
  - **진입 조건**: 사용자가 `POST /api/groups/{groupId}/join` API를 호출하여 그룹 가입에 성공했을 때 `memberships` 테이블에 레코드가 생성됩니다.
  - **상태 설명**: 사용자가 해당 그룹의 정식 멤버가 된 상태입니다.
- **DELETED (관계 종료)**

- **진입 조건**:
  - 사용자가 `DELETE /api/groups/{groupId}/leave` API를 호출하여 그룹을 탈퇴했을 때.
  - 모임장이 `DELETE /api/groups/{groupId}/members/{userId}` API를 호출하여 멤버를 강퇴했을 때.
- **상태 설명**: `memberships` 테이블에서 해당 레코드가 삭제되어 사용자와 그룹 간의 멤버십 관계가 소멸된 상태입니다.

EventAttendee (이벤트 참석)

이벤트 참석은 `status` 필드를 통해 명시적인 상태를 가집니다.

- **CONFIRMED (참석 확정)**
  - **진입 조건**: 최대 참석 인원(`maxAttendees`)에 여유가 있을 때 사용자가 참석 신청(`applyToEvent`)을 하거나, 기존 참석자가 취소하여 대기 순번이 되었을 때 진입합니다.
  - **상태 설명**: 이벤트에 참석이 확정된 상태입니다.
- **WAITING (대기 중)**
  - **진입 조건**: 최대 참석 인원이 모두 찬 상태에서 사용자가 참석 신청을 하면 진입합니다.
  - **상태 설명**: 예비 순번을 받고 대기하는 상태입니다. 기존 참석자가 취소할 경우, 신청 순서에 따라 `CONFIRMED` 상태로 자동 변경됩니다.

Post & Comment (게시글 & 댓글)

게시글과 댓글은 논리적 삭제(Soft Delete)로 생명주기를 관리합니다.

- **CREATED (생성)**
  - **진입 조건**:
    - Post: `POST /api/groups/{groupId}/posts` API 호출 시 게시글이 생성됩니다.
    - Comment: `POST /api/groups/{groupId}/posts/{postId}/comments` API 호출 시 댓글이 생성됩니다.
  - **상태 설명**: 콘텐츠가 작성되어 다른 사용자들이 조회할 수 있는 상태이며 `deleted_at`은 `NULL` 입니다.
- **MODIFIED (수정)**
  - **진입 조건**:

        - Post: 작성자가 `PATCH /api/groups/{groupId}/posts/{postId}` API를 호출하여 내용을 수정했을 때.
        - Comment: 작성자가 `PATCH /api/groups/{groupId}/posts/{postId}/comments/{commentId}` API를 호출하여 내용을 수정했을 때.
    - **상태 설명**: 기존 콘텐츠의 내용이 변경된 상태이며 `updated_at` 필드가 갱신됩니다.
- **DELETED (삭제)**
    - **진입 조건**:
        - Post: `DELETE /api/groups/{groupId}/posts/{postId}` API 호출 시.
        - Comment: `DELETE /api/groups/{groupId}/posts/{postId}/comments/{commentId}` API 호출 시.
    - **상태 설명**: 콘텐츠의 `deleted_at` 필드에 삭제 시간이 기록됩니다. `@Where` 어노테이션을 통해 삭제된 데이터는 기본 조회 결과에서 제외됩니다.

1. `USERS` 와 `GROUPS` (모임장 관계)

관계: 1:N (일대다)

설명:

- `1` (USERS) : `N` (GROUPS)
- 한 명의 회원(`USER`)은 여러 그룹(`GROUPS`)을 생성할 수 있습니다.
- 하나의 그룹(`GROUP`)은 반드시 한 명의 모임장(owner)을 가집니다.

2. `CATEGORIES` 와 `GROUPS`

관계: 1:N (일대다)

설명:

- `1` (CATEGORIES) : `N` (GROUPS)
- 하나의 카테고리(`CATEGORY`)는 여러 그룹(`GROUPS`)을 포함할 수 있습니다.
- 하나의 그룹(`GROUP`)은 반드시 하나의 카테고리(`CATEGORY`)에 속합니다.

3. `USERS` 와 `GROUPS` (멤버십 관계)

관계: N:M (다대다)

설명:

- `N` (USERS) : `M` (GROUPS)
- 한 명의 회원(`USER`)은 여러 그룹(`GROUPS`)에 가입할 수 있습니다.
- 하나의 그룹(`GROUP`)은 여러 회원(`USERS`)을 멤버로 가질 수 있습니다.
- `MEMBERSHIPS` 중간 테이블을 통해 `1:N` 관계로 해결합니다.

4. `USERS` 와 `POSTS`

관계: 1:N (일대다)

설명:

- `1` (USERS) : `N` (POSTS)
- 한 명의 회원(`USER`)은 여러 개의 게시글(`POSTS`)을 작성할 수 있습니다.
- 하나의 게시글(`POST`)은 반드시 한 명의 회원(`USER`)에 의해서만 작성됩니다.

5. `GROUPS` 와 `POSTS`

관계: 1:N (일대다)

설명:

- `1` (GROUPS) : `N` (POSTS)
- 하나의 그룹(`GROUP`)은 여러 개의 게시글(`POSTS`)을 가질 수 있습니다.
- 하나의 게시글(`POST`)은 반드시 하나의 그룹(`GROUP`)에만 속합니다.

6. `POSTS` 와 `COMMENTS`

관계: 1:N (일대다)

설명:

- `1` (POSTS) : `N` (COMMENTS)
- 하나의 게시글(`POST`)에는 여러 개의 댓글(`COMMENTS`)이 달릴 수 있습니다.
- 하나의 댓글(`COMMENT`)은 반드시 하나의 게시글(`POST`)에만 속합니다.

7. `USERS` 와 `COMMENTS`

관계: 1:N (일대다)

설명:

- `1` (USERS) : `N` (COMMENTS)
- 한 명의 회원(`USER`)은 여러 개의 댓글(`COMMENTS`)을 작성할 수 있습니다.
- 하나의 댓글(`COMMENT`)은 반드시 한 명의 회원(`USER`)에 의해서만 작성됩니다.

8. `COMMENTS` 와 `COMMENTS` (대댓글 관계)

관계: 1:N (일대다) - 자기 참조 관계

설명:

- `1` (부모 COMMENT) : `N` (자식 COMMENTS)
- 하나의 댓글(`COMMENT`)은 여러 개의 대댓글(자식 `COMMENTS`)을 가질 수 있습니다.

9. `GROUPS` 와 `EVENTS`

관계: 1:N (일대다)

설명:

- `1` (GROUPS) : `N` (EVENTS)
- 하나의 그룹(`GROUP`)은 여러 개의 일정(`EVENTS`)을 생성할 수 있습니다.
- 하나의 일정(`EVENT`)은 반드시 하나의 그룹(`GROUP`)에만 속합니다.

10. `USERS` 와 `EVENTS` (주최 관계)

관계: 1:N (일대다)

설명:

- `1` (USERS) : `N` (EVENTS)
- 한 명의 회원(`USER`)은 여러 일정(`EVENTS`)을 주최할 수 있습니다.
- 하나의 일정(`EVENT`)은 반드시 한 명의 주최자(host)를 가집니다.

11. `USERS` 와 `EVENTS` (참석 관계)

관계: N:M (다대다)

설명:

- `N` (USERS) : `M` (EVENTS)

- 한 명의 회원(`USER`)은 여러 일정(`EVENTS`)에 참석할 수 있습니다.
- 하나의 일정(`EVENT`)에는 여러 회원(`USERS`)이 참석할 수 있습니다.
- `EVENT_ATTENDEES` 중간 테이블을 통해 `1:N` 관계로 해결합니다.

12. `USERS` 와 `GROUPS` (찜 관계)

관계: N:M (다대다)

설명:

- `N` (USERS) : `M` (GROUPS)
- 한 명의 회원(`USER`)은 여러 그룹(`GROUPS`)을 찜할 수 있습니다.
- 하나의 그룹(`GROUP`)은 여러 회원(`USERS`)에게 찜을 받을 수 있습니다.
- `GROUP_LIKES` 중간 테이블을 통해 `1:N` 관계로 해결합니다.